

Laboratory Report of Nanjing University of Information Science and Technology

Experiment : Containerization with Docker

Experiment Title	Containerization with Docker	Date	2026.5.21
Class	23 人工智能	Name / ID	唐雯婷 202383910018

1. Experimental Purpose

- To learn how to package a web application into a Docker image.
- To understand immutable infrastructure and why a running container does not automatically change after source code is edited.
- To practice port mapping between the host environment and the container.
- To observe container isolation, resource usage, scaling, port conflicts, and cleanup operations.

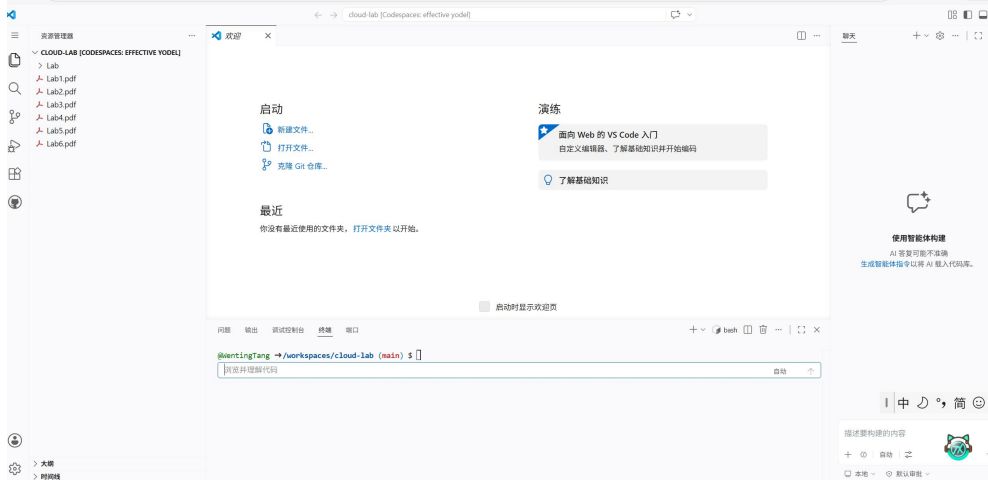
2. Experimental Environment

Platform	GitHub Codespaces / Cloud development environment
Operating Tool	Terminal in Codespaces
Main Software	Docker Engine, Nginx Alpine image
Project Files	index.html and Dockerfile
Network Ports	8080:80 and 8081:80

3. Experiment Content and Procedure

3.1 Environment Setup

First, I opened my GitHub repository, clicked the Code button, selected Codespaces, and created a new codespace. After the cloud development environment started, I used the integrated terminal at the bottom of the page to perform the Docker commands.



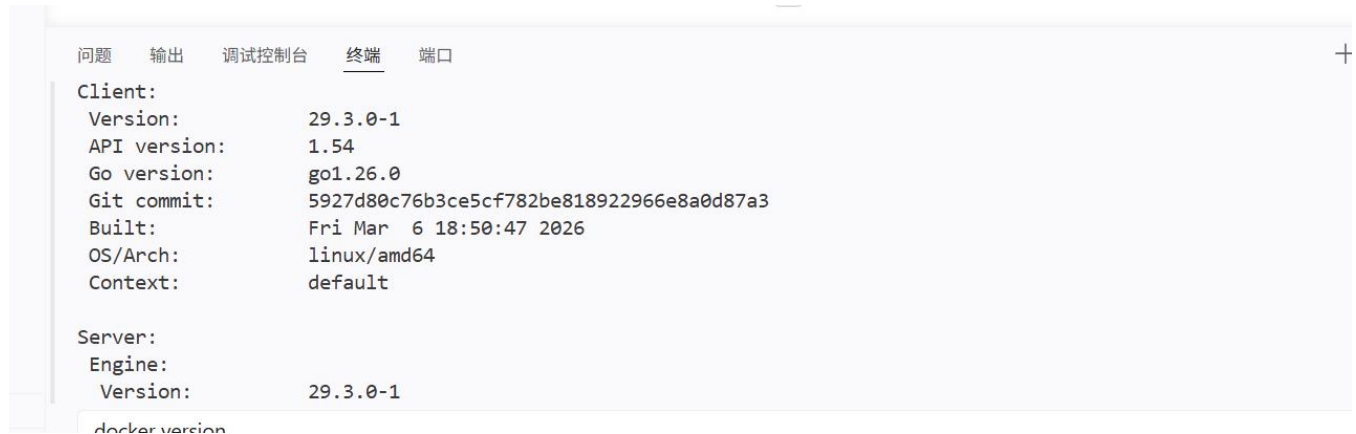
GitHub repository opened in Codespaces, with the terminal visible]

3.2 Step 1: Verify the Docker Environment

I used the following command to verify whether Docker was installed and whether the Docker daemon was running correctly:

```
docker version
```

The output showed both Client and Server version information, which means the Docker client could communicate with the Docker daemon successfully. Therefore, the environment was ready for building and running containers.



```
Client:
Version:           29.3.0-1
API version:       1.54
Go version:        go1.26.0
Git commit:        5927d80c76b3ce5cf782be818922966e8a0d87a3
Built:             Fri Mar  6 18:50:47 2026
OS/Arch:           linux/amd64
Context:           default

Server:
Engine:
Version:           29.3.0-1
```

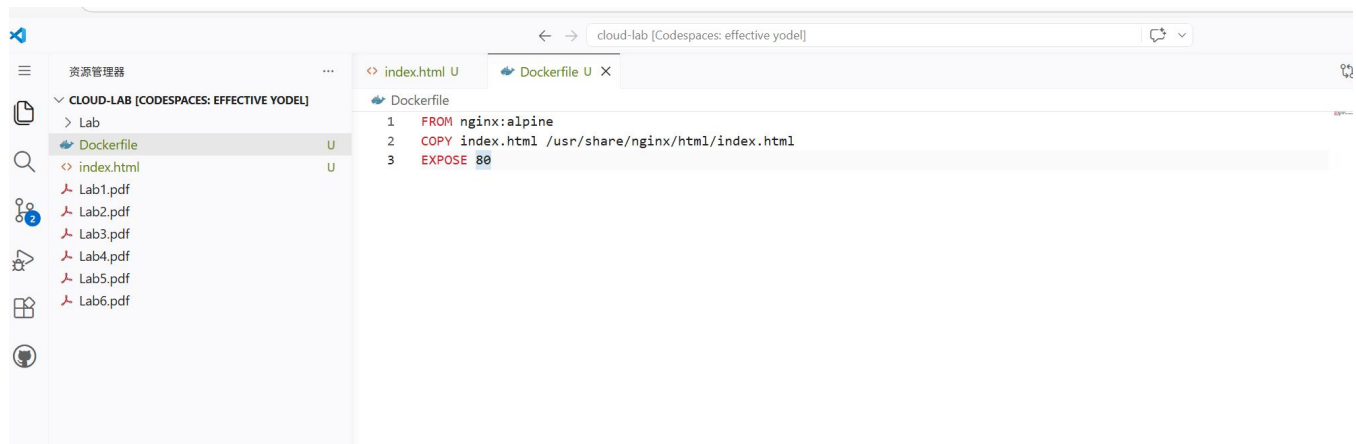
Terminal output of docker version showing Client and Server information]

3.3 Step 2: Create the Dockerfile

I created a new file named Dockerfile in the project root directory. The file name must be exactly Dockerfile, with no extension. The Dockerfile used the lightweight Nginx Alpine image as the base image, copied index.html into the default Nginx web directory, and exposed port 80 inside the container.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

This Dockerfile defines how the Docker image should be built. It packages the static web page together with the Nginx web server, so the application can run consistently in different environments.



Dockerfile created in the file explorer with the three required instructions]

3.4 Step 3: Build the Docker Image

After creating the Dockerfile, I ran the following command in the terminal to build a Docker image:

```
docker build -t cloud-lab-image:v1 .
```

The option `-t cloud-lab-image:v1` gives the image a name and version tag. The dot at the end means Docker should search for the Dockerfile in the current directory. After the build process completed, the web application was packaged into a reusable image.

```
GitCommit:
@WentingTang → /workspaces/cloud-lab (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 5.7s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 113B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 237B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd22451f848adaba14d28c7c7
=> => extracting sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5
=> => extracting sha256:94de7c34b8d25a6e2241d9f9af135eba064193ceca1b6b782e80d9d8738c24e8
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:085437c8d147f0d393fcb73b225d10adc39fdf99cff9a9975ed7688c9156d179
=> => exporting config sha256:f7e80aea03d79a860b858ad8a23a3f36ce9b0fc99710768743f288378ef22cbd
=> => exporting attestation manifest sha256:49e973ef57277531bada2b28e05aa26620bbaf634ba394918edff54d50039f9c
=> => exporting manifest list sha256:c56dc46ac52893f23bea7cac034bfadbb6c036d8b9b2f82478feaf1dcbdad456
=> => naming to docker.io/library/cloud-lab-image:v1
=> => unpacking to docker.io/library/cloud-lab-image:v1
@WentingTang → /workspaces/cloud-lab (main) $
docker version
```

Successful docker build output for cloud-lab-image:v1]

3.5 Step 4: Run the Container

Next, I launched a container from the image by using the following command:

```
docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
```

The parameter `-d` runs the container in detached mode. The port mapping `-p 8080:80` maps port 8080 of the host environment to port 80 inside the container. The option `--name my-web-container` assigns a readable name to the container.

```
@WentingTang → /workspaces/cloud-lab (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
8295c0d3fbfd727fa5e34e448d08135a63e2c000a283c821809ab7dcf8f29674
```

Successful docker run command showing the generated container ID]

3.6 Step 5: Access the Web Server

After the container started, Codespaces detected that a service was running on port 8080. I opened the forwarded port in the browser. The browser displayed my `index.html` page, which proved that the website was being served by Nginx inside the Docker container.



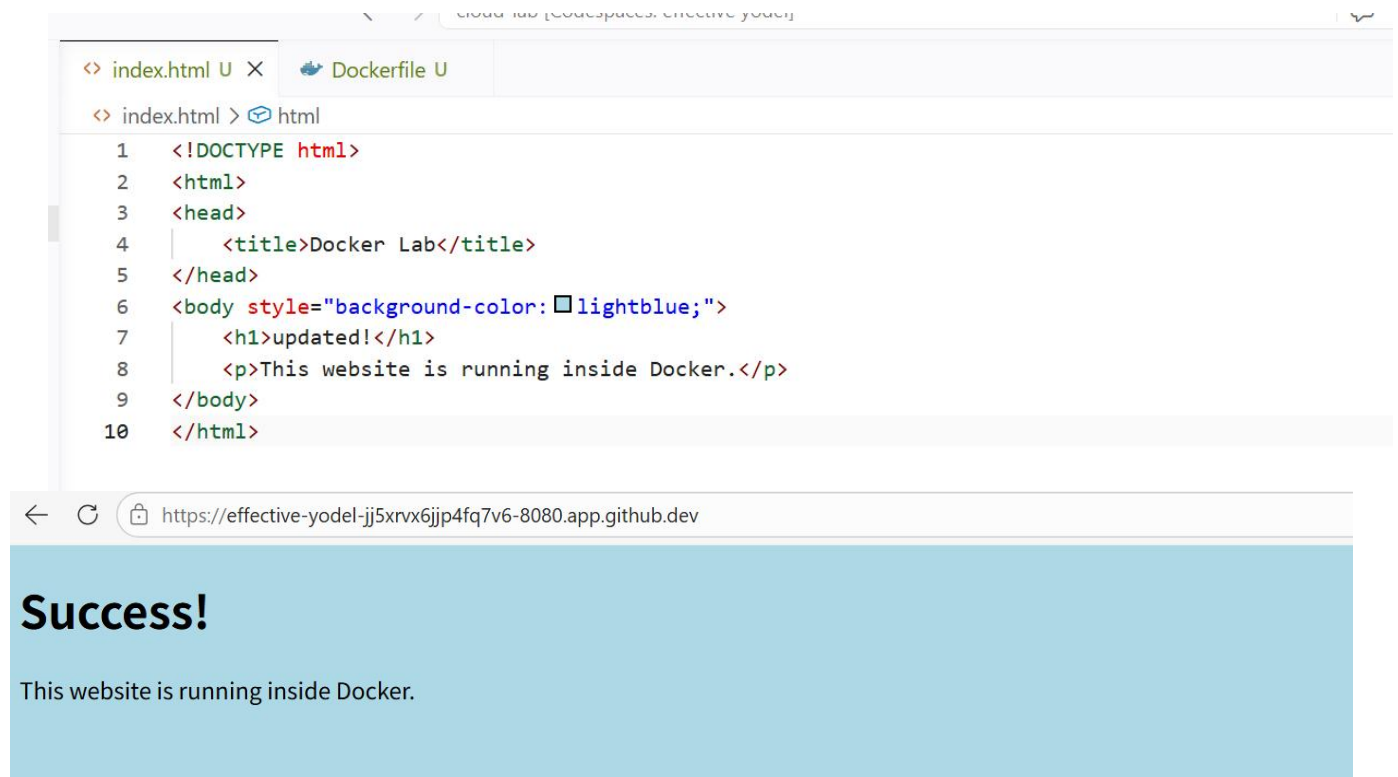
Browser page opened from port 8080 showing the personal index.html page]

3.7 Step 6: Test Container Immutability

To test immutability, I edited the source index.html file by changing part of the text or the background color. Then I refreshed the browser page that was already running on port 8080.

The website did not update immediately. This happened because the running container was created from a static Docker image. The image is a snapshot of the project at build time. Editing the source file outside the container does not modify the image or the container filesystem automatically.

This is an important feature in cloud computing. It makes deployments more reliable because the same image can be built once and run anywhere with predictable results. If changes are needed, the image should be rebuilt and the container should be restarted.



index.html modified in the editor]

Browser refreshed but still showing the old page content]

3.8 Step 7: Explore the Container Interior

A container has its own isolated filesystem. I entered the running container with the following command:

```
docker exec -it my-web-container sh
```

Inside the container, I listed the Nginx HTML directory and printed the content of index.html:

```
ls /usr/share/nginx/html/
cat /usr/share/nginx/html/index.html
exit
```

The output showed that the container had its own copy of index.html under /usr/share/nginx/html/. This proves filesystem isolation: the container environment is separated from the Codespaces project directory.

```
@WentingTang → /workspaces/cloud-lab (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
  <title>Docker Lab</title>
</head>
<body style="background-color:lightblue;">
  <h1>Success!</h1>

docker version
```

docker exec shell showing ls and cat output inside the container]

3.9 Step 8: Cloud Resource Monitoring

To observe container resource usage, I ran the following command:

```
docker stats
```

The real-time table displayed CPU percentage, memory usage, network I/O, and block I/O. The Nginx container used very little memory compared with a traditional virtual machine. This demonstrates the efficiency of containers because containers share the host operating system kernel instead of running a full guest operating system.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
8295c0d3fbfd	my-web-container	0.00%	3.41MiB / 7.758GiB	0.04%	5.01kB / 2.67kB	0B / 24.6kB	3

[Insert Screenshot: docker stats output showing CPU percentage and memory usage]

After observing the result, I pressed Ctrl+C to stop the monitoring view.

3.10 Step 9: Scaling and Port Management

To simulate scaling, I started a second container from the same image but mapped it to a different host port:

```
docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
```


Then I checked the Ports tab in Codespaces. Both 8080 and 8081 were active, and opening both ports showed identical websites. This proves that the same image can be used to quickly create multiple identical service instances.

```
^C@WentingTang → /workspaces/cloud-lab (main) docker run -d -p 8081:80 --name second-container cloud-lab-image:v1v1
7d39e59a614d45aeed1415fdd0b443179f56ffcadad62444f977e658997fa336
@WentingTang → /workspaces/cloud-lab (main) $ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
ES             7d39e59a614d     "/docker-entrypoint...." 8 seconds ago  Up 7 seconds  0.0.0.0:8081->80/tcp, [::]
ond-container  8295c0d3fbfd     "/docker-entrypoint...." 10 minutes ago Up 10 minutes  0.0.0.0:8080->80/tcp, [::]
web-container
```

[Insert Screenshot: Ports tab showing both 8080 and 8081 active]

[Insert Screenshot: Browser page opened from port 8081 showing the same website]

I also intentionally tried to run another container using host port 8080 again. Docker returned a port allocation error because port 8080 was already being used by my-web-container.

```
docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
```

```
@WentingTang → /workspaces/cloud-lab (main) $ docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
1a8aa9c5e22ceeb1a80fa1c0d31c87345549d9ae2d7c85d432bc963a29fbb96
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint thir
d-container (d0e8a20c4de2c7eb09c3871a381023e18d93ccf303ef0644854af587072cee35): Bind for 0.0.0.0:8080 failed: port is already allocated
Run 'docker run --help' for more information
```

[Insert Screenshot: Terminal error showing that port 8080 is already allocated]

This result shows that scaling is easy with Docker, but network ports must be managed correctly. Two containers cannot bind to the same host port at the same time.

3.11 Step 10: Lab Cleanup

Finally, I stopped and removed the running containers to keep the environment clean and avoid wasting cloud resources:

```
docker stop my-web-container second-container
docker rm my-web-container second-container
docker ps
```

The docker ps command showed that there were no running containers left. This cleanup step is important in cloud computing because unused resources may still consume resources or cause unnecessary costs.

```
@WentingTang → /workspaces/cloud-lab (main) $ docker stop my-web-container second-container
my-web-container
second-container
@WentingTang → /workspaces/cloud-lab (main) $ docker rm my-web-container second-container
my-web-container
second-container
@WentingTang → /workspaces/cloud-lab (main) $ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS        NAMES
@WentingTang → /workspaces/cloud-lab (main) $
```

[Insert Screenshot: docker stop, docker rm, and docker ps output confirming cleanup]

3.12 Step 11: Save Changes with Git

At the end of the lab, I saved the project changes to GitHub with the following commands:

```
git add .
git commit -m "Finished Lab 02"
git push
```

These commands added the Dockerfile and other project changes, created a commit, and pushed the work to GitHub for permanent storage.

4. Experiment Results

The experiment was completed successfully. I built a Docker image named cloud-lab-image:v1, ran a web container named my-web-container, accessed the web page through port 8080, tested immutable infrastructure, explored the isolated container filesystem, monitored container resource usage, created a second container on port 8081 to simulate scaling, observed a port conflict error, and cleaned up all running containers.

Task	Command / Action	Result
Verify Docker	docker version	Client and Server information displayed
Build image	docker build -t cloud-lab-image:v1 .	Image built successfully
Run container	docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1	Website accessible on port 8080
Test immutability	Edit index.html and refresh browser	Running site did not change automatically
Enter container	docker exec -it my-web-container sh	Container filesystem was inspected
Monitor resources	docker stats	CPU and memory usage observed
Scale service	Run second container on 8081	Two identical web services ran at the same time
Port conflict test	Run another container on 8080	Docker reported port already allocated
Cleanup	docker stop, docker rm, docker ps	No running containers remained

5. Problems Encountered and Proposed Solutions

Problem	Possible Reason	Solution
Docker command cannot connect to daemon	Docker service is not ready in Codespaces	Wait for Codespaces to finish loading, then run docker version again.
docker build cannot find Dockerfile	The Dockerfile name is wrong or not in the current directory	Use the exact name Dockerfile and run the build command in the project root directory.
Website cannot open on port 8080	The container is not running or the port is not forwarded	Run docker ps and check the Codespaces Ports tab.
Website does not update after editing index.html	The running container uses the old image snapshot	Rebuild the image and recreate the container.
Port 8080 is already allocated	Another container is already using port 8080	Use a different host port, such as 8081, or stop the old container.

6. Reflection

Through this experiment, I learned the complete workflow of containerizing a static web application with Docker. I understood that a Dockerfile describes the build process of an image, and a Docker image is a reusable package that contains the application and its runtime environment. By running the image as a container, I successfully deployed my website through Nginx and accessed it from the browser by using host-to-container port mapping.

The most important concept in this lab is immutable infrastructure. When I changed `index.html` after the container had already started, the website did not update automatically. This was not an error. It happened because the running container was based on the image created during the previous build. The image does not change unless it is rebuilt. In cloud computing, this is a useful feature because it makes deployments stable, reproducible, and easier to roll back. Engineers can build one tested image and run the same image in development, testing, and production environments.

I also learned that containers are isolated and lightweight. By entering the container shell, I saw that the container had its own filesystem. By using `docker stats`, I observed that the Nginx container consumed very few resources. Compared with traditional virtual machines, containers are more efficient because they share the host operating system kernel. Finally, the scaling and port management step helped me understand that multiple containers can be created quickly from the same image, but each service must use a valid and available host port.

7. Conclusion

In conclusion, this Docker containerization experiment helped me understand how cloud applications can be packaged, deployed, scaled, monitored, and cleaned up. The experiment connected practical Docker commands with cloud computing concepts such as immutable infrastructure, resource efficiency, isolation, scalability, and pay-as-you-go resource management. After completing this lab, I can build a Docker image, run a container, map ports, inspect a container, monitor resource usage, start multiple instances, handle port conflicts, and remove unused containers properly.

Appendix: Screenshot Checklist

- Codespaces environment and terminal
- docker version output
- Dockerfile content
- docker build success output
- docker run success output
- Web page opened on port 8080
- Modified index.html file
- Browser refresh showing unchanged page
- docker exec shell with ls and cat output
- docker stats resource table
- Ports tab showing 8080 and 8081
- Port 8081 web page
- Port conflict error
- Cleanup command output